MODULE I — CORE BRAKING ALERT

```
import random
import time

BRAKE_THRESHOLD_G = 1.2
BROADCAST_INTERVAL_MS = 100
RSSI_ALERT_THRESHOLD_DBM = -55


class SafetyPacket:
    def __init__(self, node_id, brake_status, indicator_status, relay_flag):
        self.node_id = node_id
        self.brake_status = brake_status
        self.indicator_status = indicator_status
        self.relay_flag = relay_flag

    def __str__(self):
        return (
            f"{{ node_id: {self.node_id}, brake: {self.brake_status}, "
            f"indicator: {self.indicator_status}, relay: {self.relay_flag} }}"
        )


class VehicleNode:
    def __init__(self, node_id):
        self.node_id = node_id
        self.brake_status = 0
        self.indicator_status = 0

    def read_accelerometer(self, simulate_brake=False):
        if simulate_brake:
            return round(random.uniform(1.4, 2.0), 2)
        return round(random.uniform(0.0, 0.3), 2)

    def check_braking(self, accel_g):
        self.brake_status = 1 if accel_g >= BRAKE_THRESHOLD_G else 0
        return self.brake_status

    def build_packet(self, relay_flag=0):
        return SafetyPacket(
            self.node_id,
            self.brake_status,
            self.indicator_status,
            relay_flag
        )


def simulate_rssi(close=True):
    return random.randint(-50, -40) if close else random.randint(-90, -70)


def transmit_and_receive(sender: VehicleNode, receiver_id: str, close=True):
    packet = sender.build_packet()
    rssi = simulate_rssi(close=close)

    print(f"\n[TX] Node {sender.node_id} -> broadcasting packet: {packet}")
    time.sleep(0.4)
    print("[RF] nRF24L01 channel: packet in flight...")
    time.sleep(0.4)
    print(f"[RX] Node {receiver_id} received packet. RSSI = {rssi} dBm")
    time.sleep(0.2)

    if packet.brake_status == 1 and rssi >= RSSI_ALERT_THRESHOLD_DBM:
        print(
            f"[ALERT] Node {receiver_id}: BRAKE detected at close range "
            f"(RSSI {rssi} >= {RSSI_ALERT_THRESHOLD_DBM} dBm)"
        )
        print(
            f"[OUTPUT] Node {receiver_id} -> Buzzer ON | LED: RED | "
            'LCD: "BRAKE ALERT"'
        )
    else:
        print(
            f"[OUTPUT] Node {receiver_id} -> Buzzer OFF | LED: GREEN | "
            'LCD: "SAFE"'
```

```
        )


def run_demo():
    print("=" * 60)
    print("Smart V2V Safety Broadcast System — Module I Demo")
    print("Core Braking Alert (software simulation)")
    print("=" * 60)

    vehicle_a = VehicleNode("A")

    while True:
        print("\n" + "-" * 60)
        choice = input(
            "Press [Enter] for NORMAL driving, type 'b' for SUDDEN BRAKING, "
            "or 'q' to quit: "
        ).strip().lower()

        if choice == "q":
            print("\nDemo ended.")
            break

        simulate_brake = choice == "b"
        accel = vehicle_a.read_accelerometer(simulate_brake=simulate_brake)
        print(
            f"\n[SENSOR] Node A MPU6050 reading: {accel} g "
            f"(threshold = {BRAKE_THRESHOLD_G} g)"
        )

        vehicle_a.check_braking(accel)

        if vehicle_a.brake_status == 1:
            print("[LOGIC] Deceleration exceeds threshold -> brake_status = 1")
        else:
            print("[LOGIC] Deceleration within normal range -> brake_status = 0")

        transmit_and_receive(vehicle_a, receiver_id="B", close=True)


if __name__ == "__main__":
    run_demo()
```